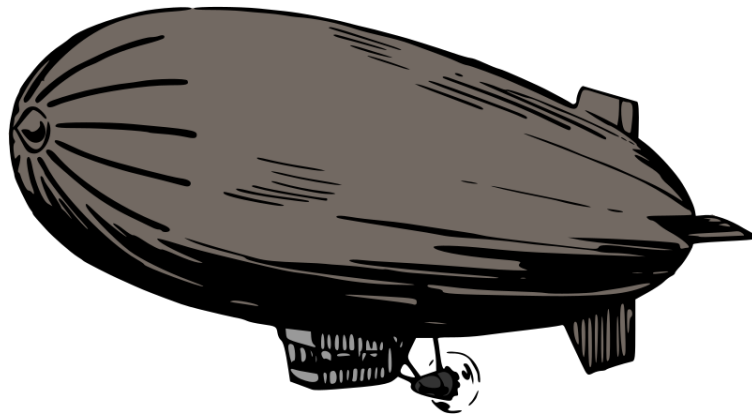


**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**ARCHITECTURAL DESIGN SPECIFICATION
CSE 4316: SENIOR DESIGN I
SUMMER 2025**



**AIRSHIP CREW
PROJECT AIRSHIP**

**LAUREN ALLBRIGHT
ARSHDEEP SINGH
CHRISTIAN MCCLAIN
BRANDON GARIBAY
AHMED OMAR**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	7.11.2025	AS	Document Creation
0.2	7.23.2025	AS, LA, CM, BG, AO	Document V1 Complete
-	-	-	-
-	-	-	-
-	-	-	-

CONTENTS

1	Introduction	5
2	System Overview	6
2.1	Ground Station Layer Description	6
2.2	Raspberry Pi Layer Description	6
2.3	Arduino Layer Description	6
3	Subsystem Definitions & Data Flow	7
4	Ground Station Layer Subsystems	8
4.1	Ground Station Subsystem 1 - MAPS API	8
4.2	Ground Station Subsystem 2 - Flask App	9
4.3	Ground Station Subsystem 3 - Front End	9
4.4	Ground Station Subsystem 4 - Network Interface	10
5	Raspberry Pi Layer Subsystems	12
5.1	Raspberry Pi Subsystem 1 - Raspberry Pi Camera	12
5.2	Raspberry Pi Subsystem 2 - Camera Vision	13
5.3	Raspberry Pi Subsystem 3 - Flask App	14
5.4	Raspberry Pi Subsystem 4 - Network Interface	15
5.5	Raspberry Pi Subsystem 5 - PID	16
5.6	Raspberry Pi Subsystem 6 - Main Controller	17
5.7	Raspberry Pi Subsystem 7 - GPS/Auxiliary Sensors	18
5.8	Raspberry Pi Subsystem 8 - Serial Interface	19
6	Arduino Layer Subsystems	21
6.1	Arduino Subsystem 1 - Serial Interface	21
6.2	Arduino Subsystem 2 - IMU & Lidar Readings	22
6.3	Arduino Subsystem 3 - Power Gauge / Current Sensor	23
6.4	Arduino Subsystem 4 - Command Interpreter	24
6.5	Arduino Subsystem 5 - Timer/Interrupt controller	25
6.6	Arduino Subsystem 6 - Motor Control	26

LIST OF FIGURES

1	Architectural layer diagram [1]	6
2	Full System Architectural Diagram [1]	7
3	Example subsystem description diagram	8
4	Example subsystem description diagram	9
5	Example subsystem description diagram	10
6	Example subsystem description diagram	11
7	Example subsystem description diagram	12
8	Example subsystem description diagram	13
9	Flask App Diagram	15
10	Network Interface Diagram	16
11	PID Diagram	17
12	Main Controller Diagram	18
13	Example subsystem description diagram	19
14	Example subsystem description diagram	20
15	Arduino Serial Interface Subsystem Diagram	21
16	IMU & Lidar Readings Subsystem Diagram	22
17	Power Gauge/Current Sensor Subsystem Diagram	23
18	Command Interpreter Subsystem Diagram	24
19	Timer/Interrupt Controller Subsystem Diagram	25
20	Motor Control System Subsystem Diagram	26

LIST OF TABLES

2	Subsystem interfaces	8
3	Subsystem interfaces	9
4	Subsystem interfaces	10
5	Subsystem interfaces	11
6	Subsystem interfaces	13
7	Subsystem interfaces	14
8	Subsystem interfaces	15
9	Subsystem interfaces	16
10	Subsystem interfaces	17
11	Subsystem interfaces	18
12	Subsystem interfaces	19
13	Subsystem interfaces	20
14	Arduino Serial Interface interfaces	22
15	IMU & Lidar Readings interfaces	23
16	Power Gauge / Current Sensor interfaces	24
17	Command Interpreter interfaces	25
18	Timer/Interrupt controller interfaces	26
19	Motor Control interfaces	26

1 INTRODUCTION

The Autonomous Cargo Blimp is an eco-friendly aerial logistics platform designed to autonomously transport cargo over short to medium distances without the need for traditional infrastructure. The system is equipped with GPS navigation, helium-based lift, electric propulsion, and safety protocols for reliable flight and landing. The scope of this project includes developing a web-based General User Interface (GUI) to communicate with the system's hardware, assembling telemetry sensors to provide the blimp's location and status, and implementing motor control to facilitate navigation and ensure safety. The key requirements include:

- Ground Station layer: Provides the user with an interface to manually control the blimp, set navigation way points, display telemetry data, and show the airship's vision and location on a map.
- Arduino layer: Extracts local sensor readings for blimp control and utilizes motor commands from the Raspberry Pi.
- Raspberry Pi layer: Retrieves ground station commands to perform motor calculations for the actuators. This layer also sends video, telemetry data, and GPS coordinates back to the ground station for updates.

2 SYSTEM OVERVIEW

This section describes the layers that describe the structure of the Autonomous Airship. The project will have three distinct layers that interact with each other to provide communication, navigation, and transportation. The layers are: Ground Station, Raspberry Pi, and Arduino layers. Each describe the device that is performing functions. The layers communicate through wireless and serial interfaces, providing data to be processed and responses to be returned.

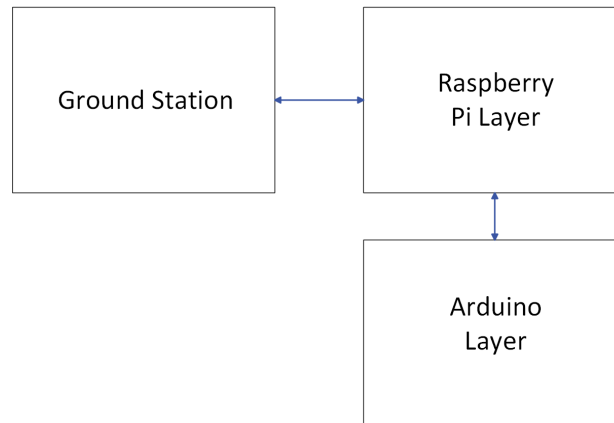


Figure 1: Architectural layer diagram [1]

2.1 GROUND STATION LAYER DESCRIPTION

The Ground Station layer deals with the user interface that is provided on a computer to control the blimp. This layer provides a front end for the user to interact with, providing a map, buttons, and fields to control the behavior of the airship. This layer also provides feedback output to the user in the form of sensor data, telemetry data, and camera output to show what is being seen through the lens of the airship. The layer has communication with the Raspberry Pi layer, which communicates with this layer to return data and receive commands, with a wireless WiFi connection. The ground station connects to the wireless network of the Raspberry Pi and communicates to the server hosted on the Pi using GET and POST commands.

2.2 RASPBERRY PI LAYER DESCRIPTION

The Raspberry Pi layer includes the main control logic of the Airship and communication interfaces between the Ground Station Layer and the Arduino layer. The two interfaces for the ground station and the Arduino are WiFi and UART/Serial respectively. The Raspberry Pi layer stores map data after user input from the ground station and performs logic to travel across that path. The Pi also receives data from the Arduino layer as well as its own auxiliary sensors that provide it with navigational data to perform actions. The Pi also performs computer vision operations to locate suitable landing spots.

2.3 ARDUINO LAYER DESCRIPTION

The Arduino layer activates actuators and collects data from sensors that it streams to the Pi Layer using serial communication. The Arduino collects data from lidar, IMU, and current sensors and provides it to the Pi. At the same time, it simultaneously receives commands and performs PWM to control motor speeds.

3 SUBSYSTEM DEFINITIONS & DATA FLOW

Figure 2 shows the complete system architecture, highlighting the layered subsystems and data flow between them. The system has three main layers: the Ground Station, the Raspberry Pi Layer, and the Arduino Layer. In the Ground Station, the front-end user interface communicates with a Flask web application, which connects to a MAPS API for route planning and interacts with the Raspberry Pi over a network. The Raspberry Pi Layer includes several key subsystems: a Flask-based backend for communication, a camera module that supplies image data to a Computer Vision module (using OpenCV), and a PID controller that assists with flight stability. A main controller manages data from auxiliary sensors like GPS and exchanges information with the Arduino through a serial interface. This layer handles high-level navigation, video processing, and external commands from the ground station. The Arduino Layer processes real-time sensor readings and controls motors. IMU and LiDAR data are handled via interrupt-driven logic, and sensor values such as power consumption are monitored by a current sensing system. These inputs are processed and sent to a command interpreter and motor control system, enabling responsive and safe flight management. Bidirectional serial links connect the Arduino to the Raspberry Pi, completing the feedback loop among sensing, computation, and actuation. This modular subsystem design allows efficient division of responsibilities, with clear data pathways between sensing, processing, and control components.

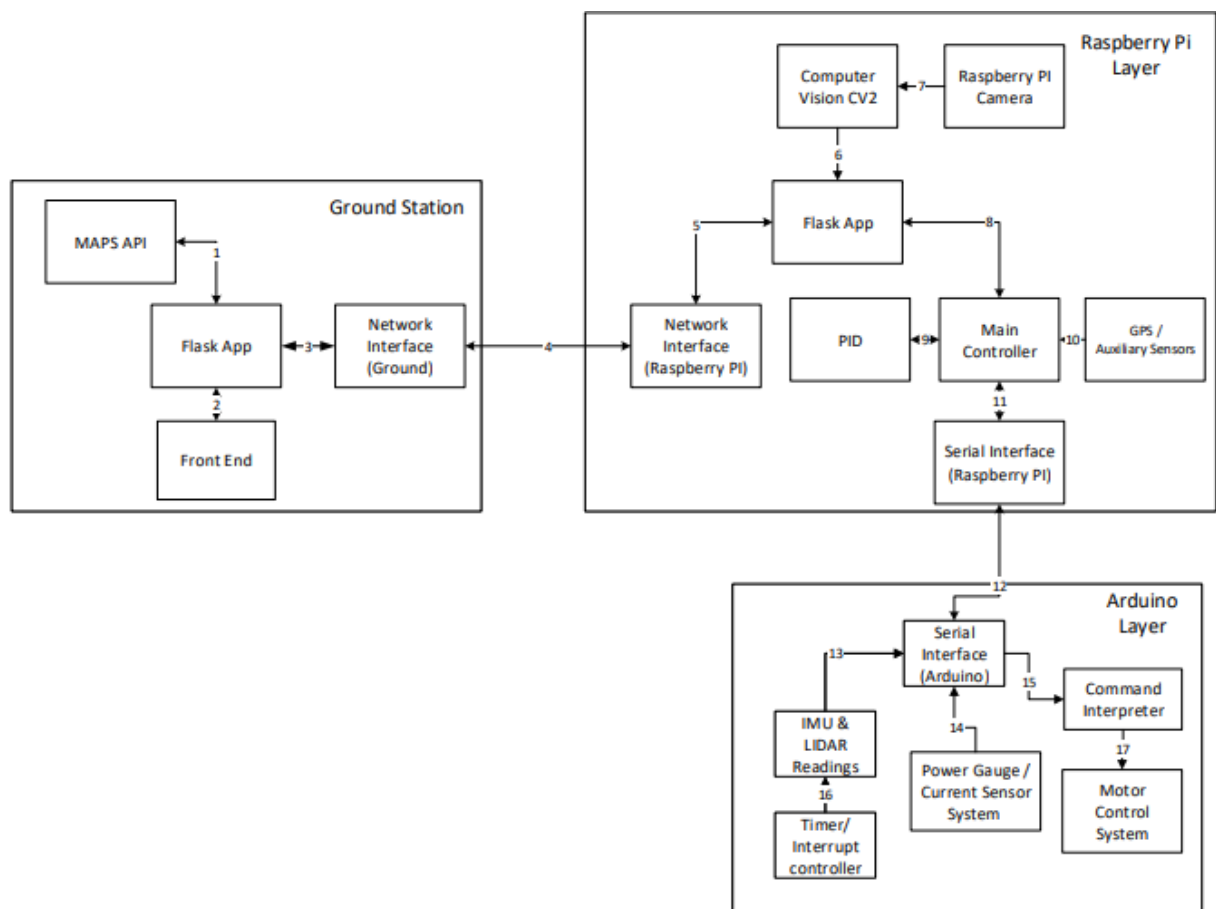


Figure 2: Full System Architectural Diagram [1]

4 GROUND STATION LAYER SUBSYSTEMS

In this section, the layer is described in some detail in terms of its specific subsystems. Describe each of the layers and its subsystems in a separate chapter/major subsection of this document. The content of each subsystem description should be similar. Include in this section any special considerations and/or trade-offs considered for the approach you have chosen.

4.1 GROUND STATION SUBSYSTEM 1 - MAPS API

This subsystem manages all mapping related functionality. Leaflet is used as a third party mapping tool to allow the user to view the blimp's position in real time and set waypoints by interacting with the map to set destination points for the blimp.

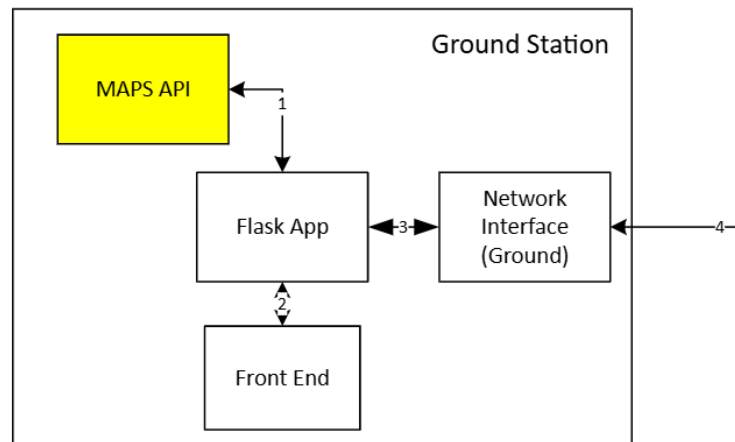


Figure 3: Example subsystem description diagram

4.1.1 ASSUMPTIONS

Leaflet is used as the client-side map rendering library. It is assumed that real time telemetry data will be available and passed through HTTP GET REQUEST from the Flask App. Waypoints will be stored in an array and sent to the Pi when the user decides.

4.1.2 RESPONSIBILITIES

This subsystem is responsible for rendering an interactive map centered on the blimp and will update the blimp's current position live during flight. The user will be able to plot waypoints to create a path for the blimp to take.

4.1.3 SUBSYSTEM INTERFACES

Table 2: Subsystem interfaces

ID	Description	Inputs	Outputs
#01	Telemetry Display	Blimp Coordinates	Blimp Marker Movement
#01	User Waypoint Input	User Click	Map Waypoint
#01	Waypoint Coordinates	User Button Click	Waypoint Coordinates

4.2 GROUND STATION SUBSYSTEM 2 - FLASK APP

The Flask App serves as the backend of the GUI. It communicates between the frontend, Map API, and network layer.

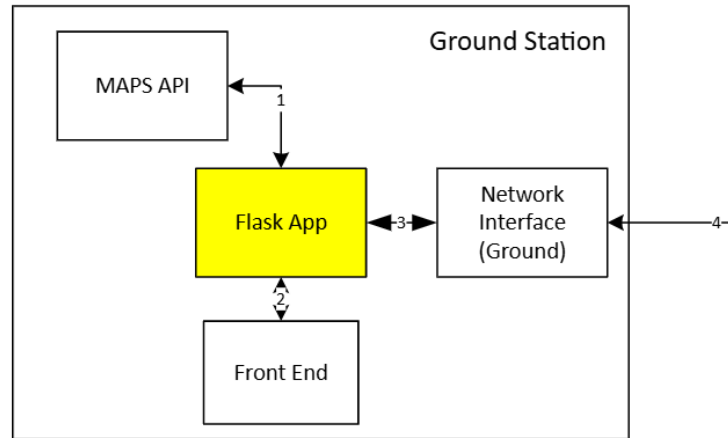


Figure 4: Example subsystem description diagram

4.2.1 ASSUMPTIONS

The flask app runs over a secure local network and all real time updates are handled through HTTP requests and responses. Flask provides endpoints for both sending and receiving data.

4.2.2 RESPONSIBILITIES

The Flask App provides routes for the sending and receiving telemetry, control commands, video, and map interactions. It mediates all communicates between the front end, map layer, and network interface.

4.2.3 SUBSYSTEM INTERFACES

Table 3: Subsystem interfaces

ID	Description	Inputs	Outputs
#03	Telemetry	Sensor Data	Blimp Data and Coordinates
#02	Control Route	User Input	Blimp Commands and Waypoints
#01	Map	Map Data	Interactive Map

4.3 GROUND STATION SUBSYSTEM 3 - FRONT END

This subsystem will provide a user interface for the ground station.

4.3.1 ASSUMPTIONS

Users will have feedback and control of the blimp through the user interface. The user can use keyboard or mouse to control the blimp manually or through waypoint assignment.

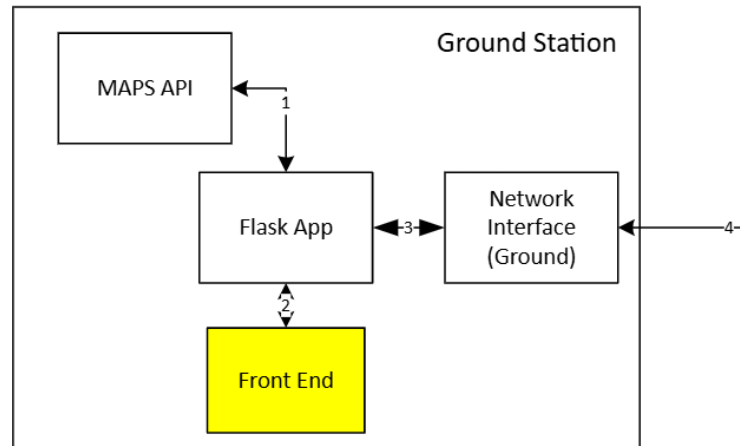


Figure 5: Example subsystem description diagram

4.3.2 RESPONSIBILITIES

The front end will display real time telemetry and allow the user to control the movement and behavior of the blimp through the graphic user interface. The leaflet map will be embedded and allow the user to plot waypoints for the blimps destination. The GUI will also display video feed from the blimps camera.

4.3.3 SUBSYSTEM INTERFACES

Table 4: Subsystem interfaces

ID	Description	Inputs	Outputs
#02	App Communication	User Input	Blimp commands
#02	Telemetry Display	Blimp Data	Updated UI
#02	Video Stream	Video Frames	Live Feed

4.4 GROUND STATION SUBSYSTEM 4 - NETWORK INTERFACE

This section should be a general description of a particular subsystem for the given layer. For most subsystems, an extract of the architectural block diagram with data flows is useful. This should consist of the subsystem being described and those subsystems with which it communicates.

The Network Interface subsystem handles direct communication between the Flask app and the Pi Layer.

4.4.1 ASSUMPTIONS

The Pi and Flask server communicate through HTTP request on a local network. Telemetry and control messages use lightweight data formats.

4.4.2 RESPONSIBILITIES

The Network interface subsystem is responsible for transmitting control commands from the flask server to the Pi and receiving telemetry and video from the Pi.

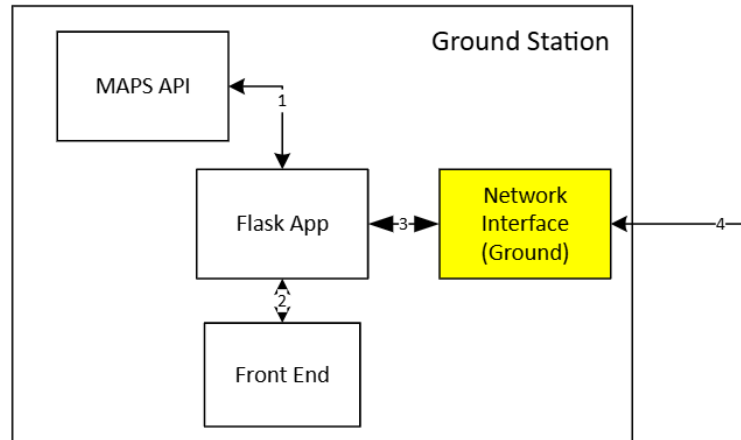


Figure 6: Example subsystem description diagram

4.4.3 SUBSYSTEM INTERFACES

Table 5: Subsystem interfaces

ID	Description	Inputs	Outputs
#03	Blimp Control	Commands	Blimp Movement
#04	Telemetry	Sensor Data	Display Blimp Data

5 RASPBERRY PI LAYER SUBSYSTEMS

The Raspberry Pi layer is responsible for the main operations of the blimp. The Pi must receive navigational commands from the Ground Station to set a course for itself, then use the data from main and auxiliary sensors to navigate the world. The Raspberry Pi performs communication between the ground station and the Arduino and performs the logic that keeps the airship on the right track and elevation.

5.1 RASPBERRY PI SUBSYSTEM 1 - RASPBERRY PI CAMERA

The Raspberry Pi Camera is crucial in the Raspberry Pi Layer, capturing live video for tasks like edge detection. Mounted on the airship, it streams RGB frames to the Computer Vision subsystem, connected via the CSI port. It works with the Flask app for data routing and processing, forming the basis of all vision-based navigation and perception.

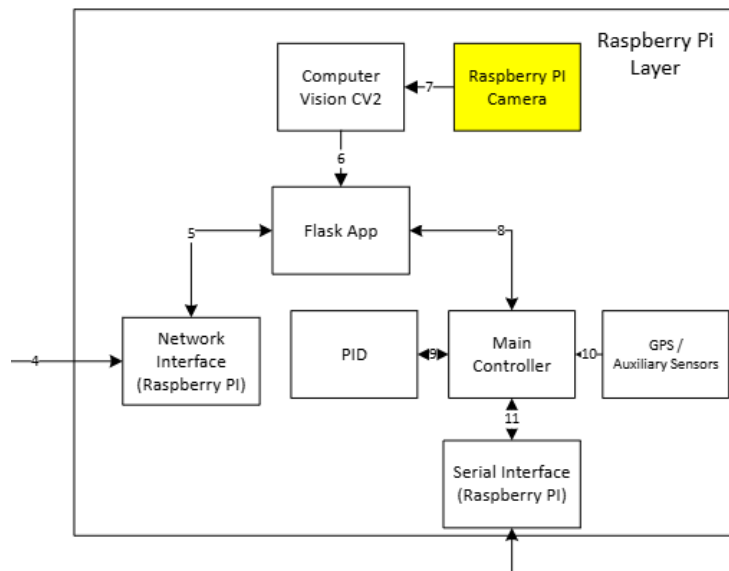


Figure 7: Example subsystem description diagram

5.1.1 ASSUMPTIONS

- The Raspberry Pi Camera is properly installed and compatible with the Raspberry Pi OS and hardware.
- The camera is configured to deliver frames at a consistent resolution and frame rate.
- Adequate lighting conditions are present in the environment to ensure clear image capture.
- The camera driver and picamera2 library are correctly installed and functional.
- The video stream is accessible to the CV2 (OpenCV) module in real-time without latency or frame drop issues.
- The camera operates continuously during flight unless explicitly shut off by a controller signal.

5.1.2 RESPONSIBILITIES

- Capture continuous RGB video frames from the surrounding environment.

- Interface directly with the picamera2 module to deliver live video feed to the processing pipeline.
- Ensure image data is formatted correctly (e.g., RGB or grayscale) for computer vision analysis.
- Maintain consistent performance (resolution and frame rate) during autonomous flight operations.
- Enable streaming or handoff of captured frames to the Flask App and OpenCV modules for further processing.

5.1.3 SUBSYSTEM INTERFACES

Table 6: Subsystem interfaces

ID	Description	Inputs	Outputs
#07	CSI Interface (Camera Serial Interface)	N/A	Raw RGB video frames
#06	CV2 Processing Module	N/A	RGB or grayscale frames sent to OpenCV

5.2 RASPBERRY PI SUBSYSTEM 2 - CAMERA VISION

The Computer Vision (CV) subsystem, part of the Raspberry Pi Layer, processes visual data from the onboard camera for real-time edge detection and object recognition, aiding autonomous navigation and obstacle avoidance. It receives raw video frames, processes them with OpenCV, and sends processed data or triggers to the Flask App and PID controller.

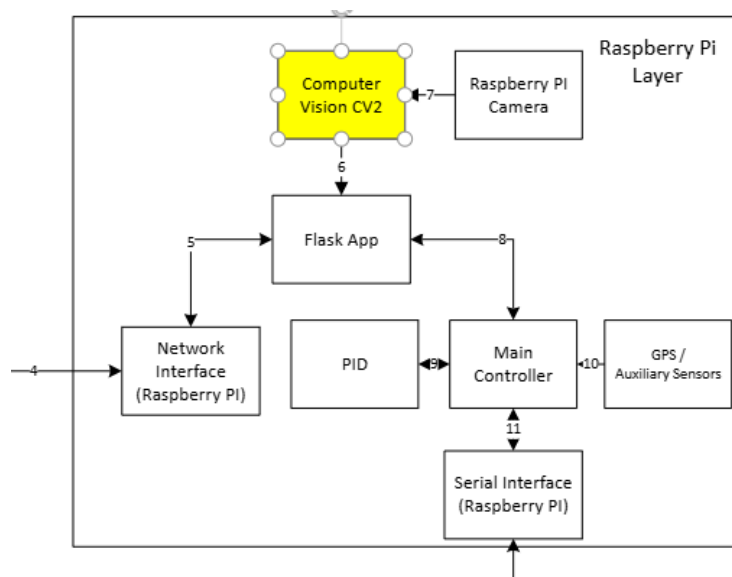


Figure 8: Example subsystem description diagram

5.2.1 ASSUMPTIONS

- The Raspberry Pi Camera works properly and consistently provides video frames at a fixed resolution and frame rate.
- The environment offers enough lighting and contrast for reliable edge detection.
- The OpenCV setup is fully installed and integrated with the Python-based Flask application.
- The Flask App ensures consistent routing and access to processed image data.
- The subsystem does not control actuators directly; instead, it informs navigation decisions through data passed to other modules such as PID or Main Controller.
- Network latency and processing times are acceptable for real-time decision-making.

5.2.2 RESPONSIBILITIES

- Receive raw RGB frames from the Raspberry Pi Camera.
- Convert RGB frames to grayscale and apply edge detection (e.g., Canny algorithm).
- Optionally, process frames for object recognition, bounding boxes, or tracking (future development).
- Stream processed video frames or analytics to the Flask app for UI or logging purposes.
- Provide navigation-related data (e.g., edge map or obstacle detection flags) to the PID controller or main controller.
- Operate autonomously without user input during flight mode.

5.2.3 SUBSYSTEM INTERFACES

Table 7: Subsystem interfaces

ID	Description	Inputs	Outputs
#07	Video feed from Raspberry Pi Camera	Raw RGB image frames	Image frames to be processed
#07	Flask App interface (internal API route)	Navigation request or trigger	Processed frames or edge data

5.3 RASPBERRY PI SUBSYSTEM 3 - FLASK APP

The Flask App is responsible for hosting the web server on the pi which becomes the central unit for the networking and data transmission between the Pi and Ground station. The flask app interacts with the routes to send and receive GET and POST requests that allow the Ground Station and Pi to communicate.

5.3.1 ASSUMPTIONS

The Flask App assumes a proper network interface is set up so that the Pi communicate with the Ground Station, it also assumes that the Flask Routes contain correct and proper information to be send to the Ground Station.

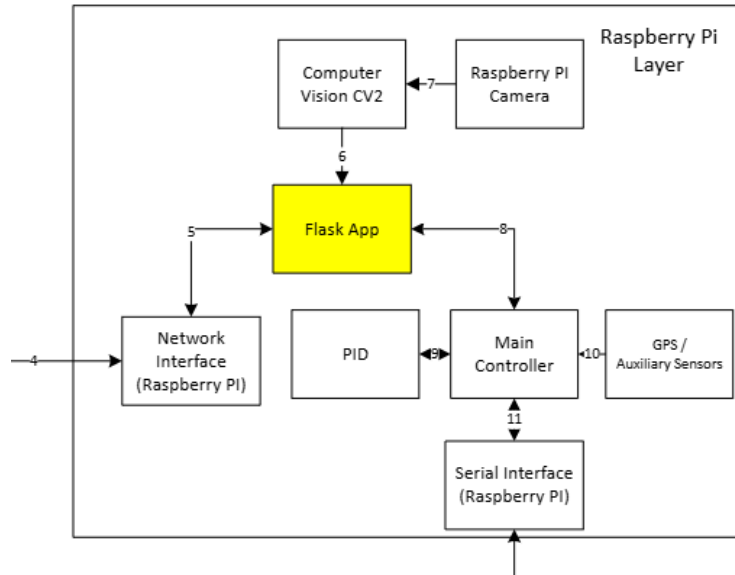


Figure 9: Flask App Diagram

5.3.2 RESPONSIBILITIES

The Flask app communicates with the Ground Station via the Network Interface and handles GET and POST request for the many routes. The app also handles the video streaming from the Pi's camera to the ground station.

5.3.3 SUBSYSTEM INTERFACES

Table 8: Subsystem interfaces

ID	Description	Inputs	Outputs
#05	Network Traffic	Command and Navigation Instructions	Sensor and Video Data
#06	Video Frames for Streaming	N/A	Video Frames
#08	Sensor Data, Command, and Navigational data	Sensor Data	Command and Navigation Data

5.4 RASPBERRY PI SUBSYSTEM 4 - NETWORK INTERFACE

The network interface is the subsystem that allows the Pi to communicate on a wireless network and communicate with the ground station.

5.4.1 ASSUMPTIONS

The assumptions for the proper functions for the network interface are that the hardware and the medium that the network interface interacts with is working correctly. Additionally, the other devices must be connected to the network.

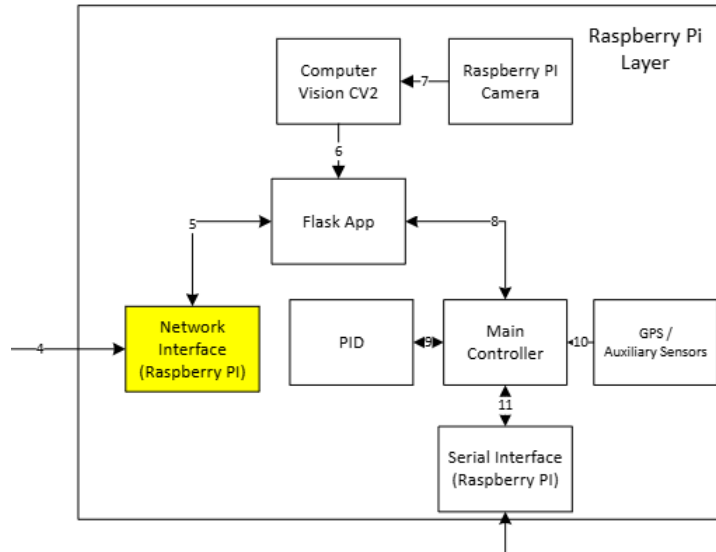


Figure 10: Network Interface Diagram

5.4.2 RESPONSIBILITIES

Creates a wireless access point for other devices to connect to and hosts a static IP so that other devices can reliably connect to the Airship and control it. Transmit network traffic to and from the airship.

5.4.3 SUBSYSTEM INTERFACES

Table 9: Subsystem interfaces

ID	Description	Inputs	Outputs
#05	Network Traffic sent to the main controller	N/A	Commands and Navigational Data
#04	Network Traffic from Ground Station	Commands and Navigational Data	N/A

5.5 RASPBERRY PI SUBSYSTEM 5 - PID

The PID system helps the Airship stay at a constant altitude. The system prevents the airship from moving up or down from a certain altitude set by the Ground Station.

5.5.1 ASSUMPTIONS

The sensor input to the PID system is correct and non-drifting. The PID coefficients (K_i , K_d , and K_p) are tuned correctly to perform the actions that are required to stabilize the system.

5.5.2 RESPONSIBILITIES

The PID subsystem takes in Altitude measurements provided by the IMU and the Auxiliary sensor and uses it to calculate an output that would control the motor speed of upward facing motors. The output would allow the motors to speed up or slow down to keep the airship at a set altitude.

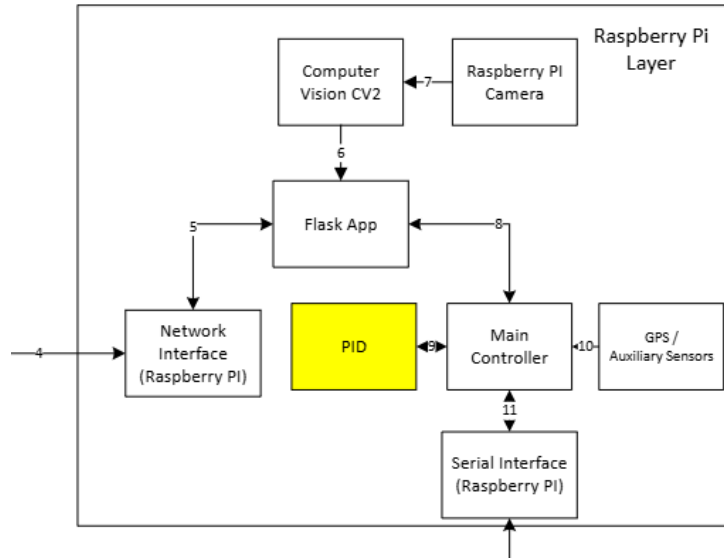


Figure 11: PID Diagram

5.5.3 SUBSYSTEM INTERFACES

Table 10: Subsystem interfaces

ID	Description	Inputs	Outputs
#09	PID Input / Output	Altitude	PID output

5.6 RASPBERRY PI SUBSYSTEM 6 - MAIN CONTROLLER

The main controller performs the major logic that helps navigate the airship. It also controls the threads of all other subsystems and processes on the Raspberry Pi. It acts as a bridge between the Ground Station and Arduino.

5.6.1 ASSUMPTIONS

The data into the main controller is correct and is updated regularly. The output of the main controller is accurately and rapidly sent to the actuators of the airship.

5.6.2 RESPONSIBILITIES

Takes in geographical data from the Ground Station and data from the sensors and creates outputs for the BLDC motors to navigate to those way-points. The main controller also controls the threads of all other systems and makes sure all they are all online and running. The main controller also facilitates communication between subsystems.

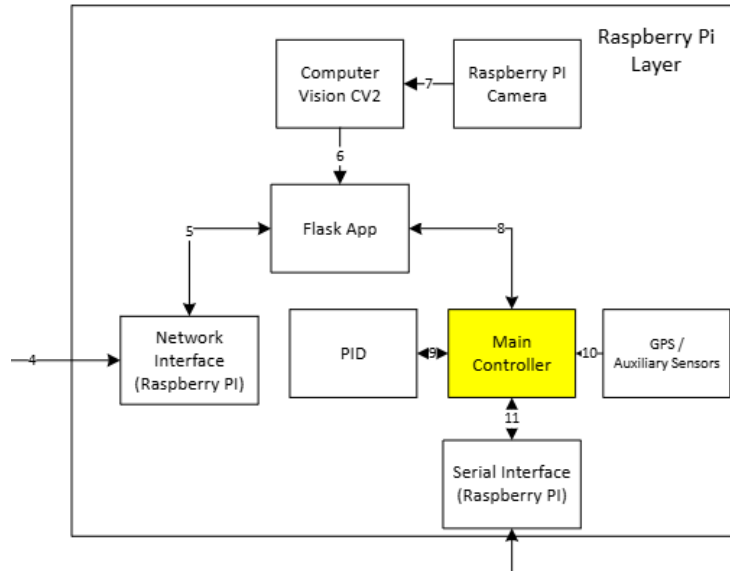


Figure 12: Main Controller Diagram

5.6.3 SUBSYSTEM INTERFACES

Table 11: Subsystem interfaces

ID	Description	Inputs	Outputs
#08	Command and Navigation Data from the Flask App	Forwarded Data from Flask	Sensor Data
#09	PID output and Altitude Input	Altitude Input	PID output
#10	GPS and Aux Sensor Data from GPS module	GPS data	N/A
#11	Communication Bus between Arduino and Pi	Sensor Data	Motor Commands

5.7 RASPBERRY PI SUBSYSTEM 7 - GPS/AUXILIARY SENSORS

This section describes the assumptions, responsibilities, and subsystem interfaces of the GPS/Auxiliary Sensors subsystem. The subsystem acquires precise telemetry data, GPS coordinates, and other aviation data to report the flight status of the blimp. The data is collected via global satellites to ensure accurate readings using the GPS hat device.

5.7.1 ASSUMPTIONS

The GPS/Auxiliary subsystem assumes that the GPS hat is capable of maintaining a constant fix on the satellites. The assumption is essential for the system's navigation; To guarantee a fix, the GPS hat will be connected to an external antenna. The subsystem will also contain code that refreshes data if a fix is lost during flight.

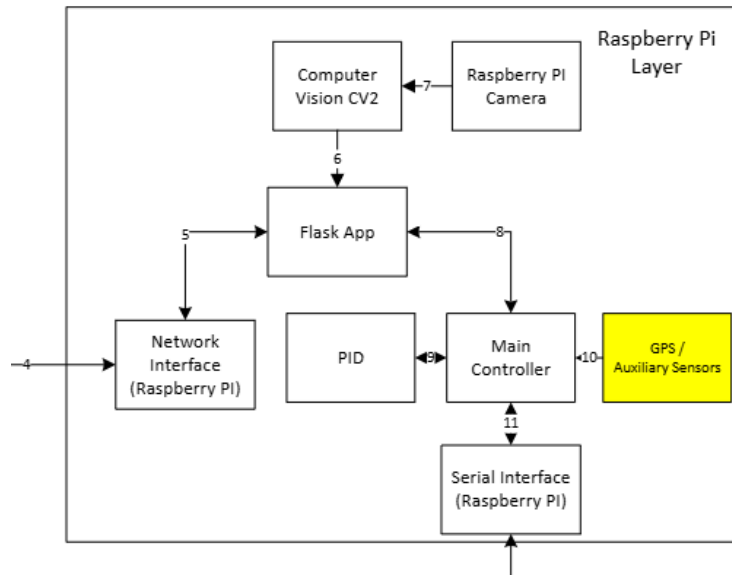


Figure 13: Example subsystem description diagram

5.7.2 RESPONSIBILITIES

The subsystem is responsible for relaying critical aviation data to the Ground Station. Understanding the blimp's location and other telemetry information will assist in autonomous navigation and flight status, allowing for the correction of any errors. Additionally, the GPS/Auxiliary Sensors subsystem will significantly aid in following the Federal Aviation Administration's guidelines for drone operation.

5.7.3 SUBSYSTEM INTERFACES

Each of the inputs and outputs for the subsystem are defined here. The subsystem has one output from the GPS/Auxiliary Sensor that feeds to the Main Controller subsystem.

Table 12: Subsystem interfaces

ID	Description	Inputs	Outputs
#10	Sends aviation data to the main controller	N/A	GPS and Auxiliary Data

5.8 RASPBERRY PI SUBSYSTEM 8 - SERIAL INTERFACE

This subsystem is the serial interface that allows the Pi to communicate directly with the Arduino and provide crucial data.

5.8.1 ASSUMPTIONS

The main assumptions in the performance of the serial interface are that the interface works at the same baud rate as the Arduino Serial Interface and that the data to be transmitted is correct.

5.8.2 RESPONSIBILITIES

This subsystem sends the motor control commands from the main controller to the Arduino and receives and forwards sensor data from the Arduino to the main controller in a timely and rapid manner.

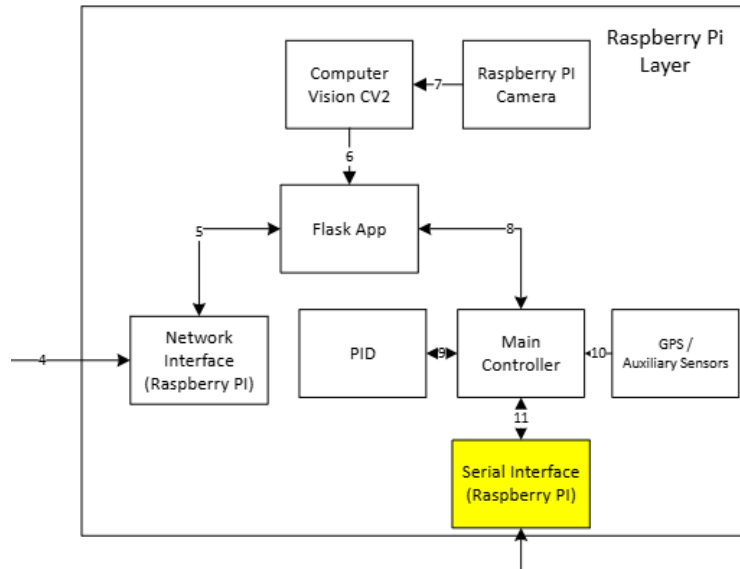


Figure 14: Example subsystem description diagram

5.8.3 SUBSYSTEM INTERFACES

Table 13: Subsystem interfaces

ID	Description	Inputs	Outputs
#11	Pi and Arduino communication line	Sensor Data	Motor Commands
#12	UART Interface	UART RX	UART TX

6 ARDUINO LAYER SUBSYSTEMS

The Arduino layer is mainly responsible for collecting sensor data and controlling BLDC motors connected to the Arduino board. The Layer receives commands from the Raspberry Pi layer and sends back info using the Serial Communication sub system and collects info from various sensors.

6.1 ARDUINO SUBSYSTEM 1 - SERIAL INTERFACE

This subsystem is responsible for the inter-layer communication between the Raspberry Pi and the Arduino. The layer is a UART that receives asynchronous commands from the Pi at the baud rate of 115200 and provides the Pi, periodically, data from the sensors connected to this subsystem.

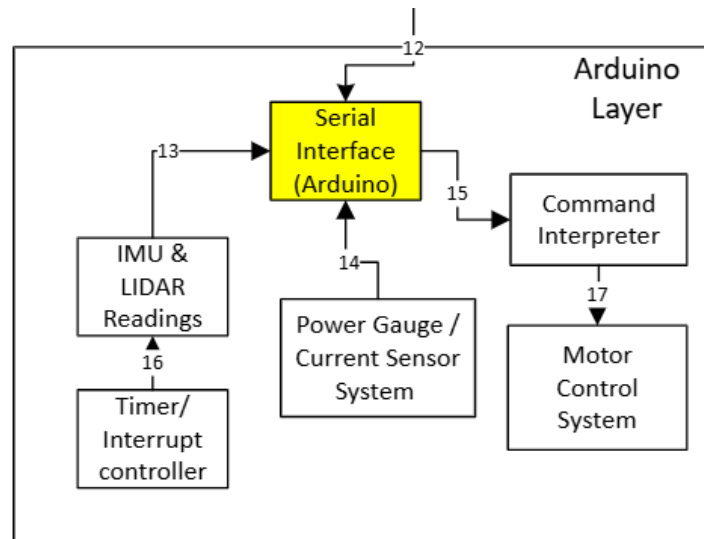


Figure 15: Arduino Serial Interface Subsystem Diagram

6.1.1 ASSUMPTIONS

The assumption is that the UART modules on the Arduino and Raspberry Pi are able to communicate properly and with the same baud rate.

6.1.2 RESPONSIBILITIES

This subsystem should send data from the Lidar, IMU, and current gauge to the Raspberry Pi. The system must periodically send this data and also receive commands from the Raspberry Pi.

6.1.3 SUBSYSTEM INTERFACES

The system has 2 inputs from the sensor and a input from the Raspberry Pi layer and a output.

Table 14: Arduino Serial Interface interfaces

ID	Description	Inputs	Outputs
#12	UART input and output with Raspberry Pi	motor command	sensor data
#13	Sensor data from IMU & Lidar sensor	Transmit Data Signal	Telemetry and Distance Data
#14	Current Flow data from current sensor	Current Flow Data	N/A
#15	Command message from Raspberry Pi	N/A	motor speed command

6.2 ARDUINO SUBSYSTEM 2 - IMU & LIDAR READINGS

The IMU and LIDAR Readings subsystem retrieves raw data from the sensors and formats the relevant data to facilitate data extraction for the Raspberry Pi. The data is periodically sent to the receiver at a fixed interval to improve overall system performance.

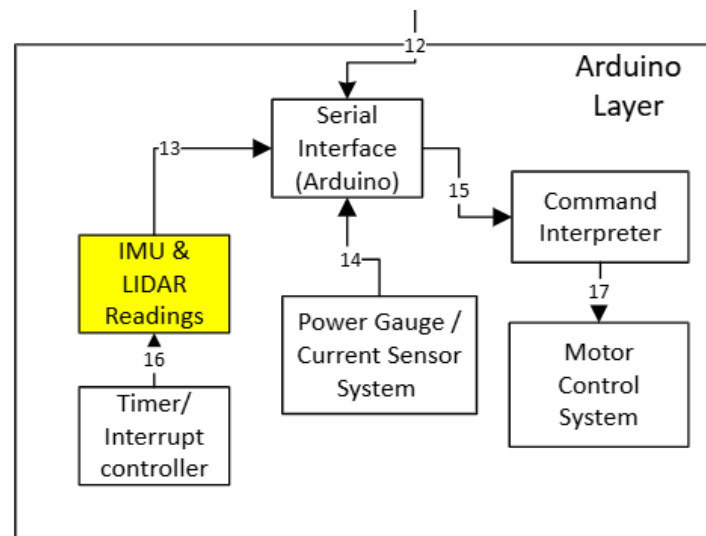


Figure 16: IMU & Lidar Readings Subsystem Diagram

6.2.1 ASSUMPTIONS

The subsystem expects the sensors to be fully functioning for adequate system operation. To ensure proper functionality, the subsystem performs precise calibration upon startup and has built-in contingencies to notify the user if an issue arises. Moreover, the sensors utilize the I2C communication protocol to synchronize with the Arduino. The use of the protocol ensures reliable data transmission through a simple and low-cost implementation.

6.2.2 RESPONSIBILITIES

IMU and LIDAR Readings play a supplementary role in the system's orientation, motion, and obstacle detection. The LIDAR informs the Raspberry Pi of the distance between the sensor and an obstacle. This

function provides additional proximity sensing to the airship's vision system, which improves reliability and safety. The IMU also supports the GPS/Auxiliary subsystem by providing the Raspberry Pi with its speed, angle, and direction with respect to the Earth's magnetic field.

6.2.3 SUBSYSTEM INTERFACES

Each of the inputs and outputs for the subsystem are defined here. The subsystem has one input from the Timer/Interrupt Controller and one output for the Arduino Serial Interface.

Table 15: IMU & Lidar Readings interfaces

ID	Description	Inputs	Outputs
#13	Sensor data from the IMU and LIDAR sensors	Transmit Data Signal	Telemetry and Distance Data
#16	Interrupt from the Timer/Interrupt Controller	N/A	Transmit Data Signal

6.3 ARDUINO SUBSYSTEM 3 - POWER GAUGE / CURRENT SENSOR

This subsystem is providing battery power / current data. This data will be forwarded to the Raspberry Pi to make command decisions.

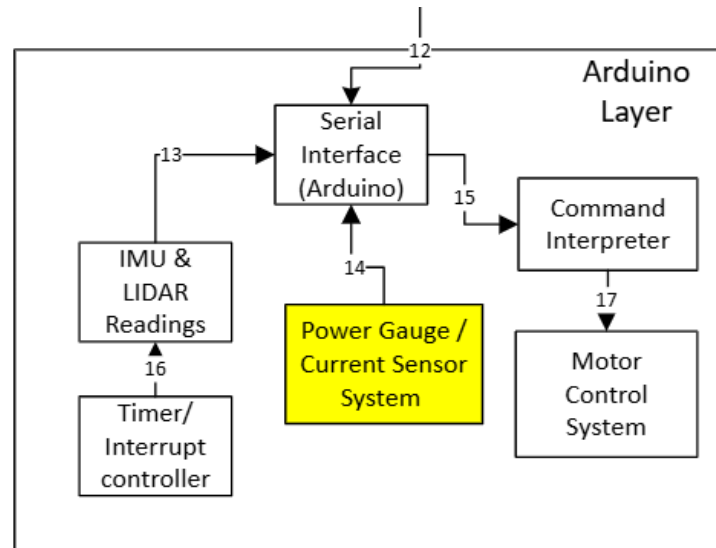


Figure 17: Power Gauge/Current Sensor Subsystem Diagram

6.3.1 ASSUMPTIONS

The current sensor can accurately measure the current flowing and allow us to measure the power left in the battery.

6.3.2 RESPONSIBILITIES

Measures the current flowing through the system to estimate the remaining battery.

6.3.3 SUBSYSTEM INTERFACES

Table 16: Power Gauge / Current Sensor interfaces

ID	Description	Inputs	Outputs
#14	Current sensor data to serial interface	N/A	Current Flow Rate

6.4 ARDUINO SUBSYSTEM 4 - COMMAND INTERPRETER

This subsystem is responsible for parsing command received from the serial interface and directing those commands to motor speed control.

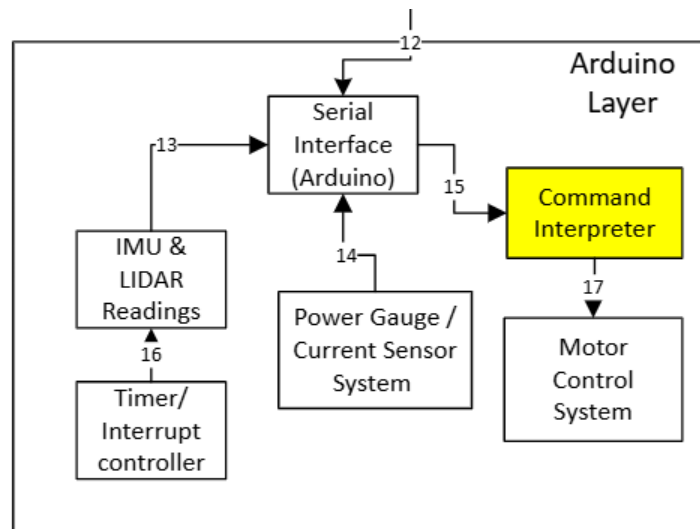


Figure 18: Command Interpreter Subsystem Diagram

6.4.1 ASSUMPTIONS

Commands sent by the serial interface and those sent by the Raspberry pi and are assumed to be commanded by the interpreter.

Another assumption is that the command interpreter is able to reach the components the command is controlling.

6.4.2 RESPONSIBILITIES

This subsystem is responsible for receiving commands from the serial interface for the motor control. This subsystem breaks down the commands into two sections called key and value and it directs those commands to the appropriate motor control.

6.4.3 SUBSYSTEM INTERFACES

Table 17: Command Interpreter interfaces

ID	Description	Inputs	Outputs
#15	Command from serial interface	Command	N/A
#17	Output to motor control system	N/A	key/value

6.5 ARDUINO SUBSYSTEM 5 - TIMER/INTERRUPT CONTROLLER

The Timer/Interrupt Controller subsystem is responsible for providing the IMU and LIDAR Readings subsystem with an interrupt signal that conveys when to send the sensor data to the Raspberry Pi.

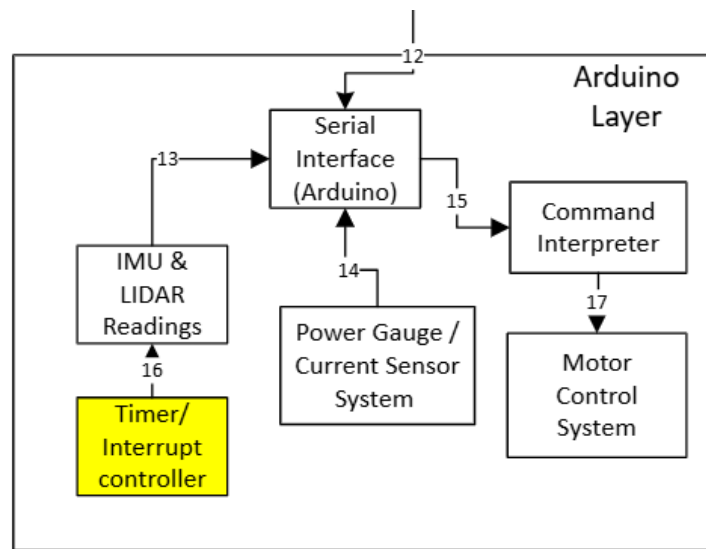


Figure 19: Timer/Interrupt Controller Subsystem Diagram

6.5.1 ASSUMPTIONS

The subsystem expects the Arduino's System Clock Frequency to be generated upon boot. The System Clock is essential for the subsystem to work as it performs calculations on the frequency to output precise interrupt cycles.

6.5.2 RESPONSIBILITIES

This subsystem is vital for the IMU and LIDAR Readings subsystem because the sensors rely solely on the interrupts to output their data. The Timer/Interrupt Controller subsystem also enhances overall system performance by reducing the need for constant data polling. The effect of this subsystem enables the Arduino to retrieve commands more quickly, allowing it to control the propeller motors more efficiently.

6.5.3 SUBSYSTEM INTERFACES

Each of the inputs and outputs for the subsystem are defined here. The subsystem has one output for the IMU and LIDAR Reading subsystem.

Table 18: Timer/Interrupt controller interfaces

ID	Description	Inputs	Outputs
#16	Interrupt from the Timer/Interrupt Controller	N/A	Transmit Data Signal

6.6 ARDUINO SUBSYSTEM 6 - MOTOR CONTROL

This subsystem is responsible for controlling the speeds of the BLDC motors of the airship.

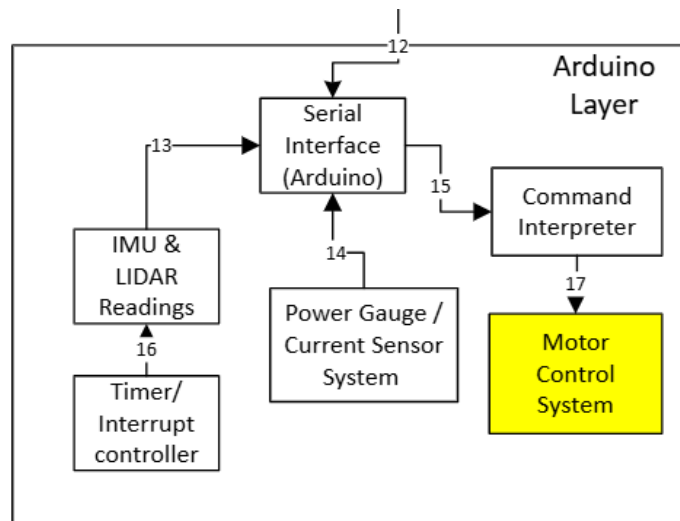


Figure 20: Motor Control System Subsystem Diagram

6.6.1 ASSUMPTIONS

Correct commands are sent from the command interpreter to control the motor speed. The motors connected must also be properly operating.

6.6.2 RESPONSIBILITIES

This subsystem is responsible for motor speeds. It uses PWM to control four BLDC motors on the blimp to provide lift and thrust.

6.6.3 SUBSYSTEM INTERFACES

Table 19: Motor Control interfaces

ID	Description	Inputs	Outputs
#17	Motor control speed command	key/value	N/A

REFERENCES

- [1] Microsoft Corporation. Visio in Microsoft 365: Diagramming and visual collaboration. Accessed: [July, 7th, 2025].